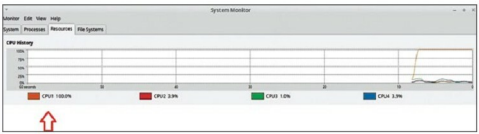

Figure 4: Serial Application performance monitoring system monitor

We are executing the applications on a system with Intel CoreTM i5-3450 CPU @3.10 GHz x 4, 4GB RAM running a Linux Mint 17.2 Rafaela 32-bit operating system. We can use the *htop* command and system monitor to demonstrate the resource utilisation of the application, as shown in Figures 3 and 4. It is very clear that, of the four cores available, the application is running on Core/CPU 1 as it is a serial application with one thread of control. The other cores are under-utilised because of this. With only 27.4 per cent of CPU utilisation, the application failed to take the advantage of its four cores, so the performance of the application is very poor.

Multi-threaded (parallel) application performance monitoring

Matrix multiplication of two NxN matrices can be done in parallel, in many ways. We considered doing it in parallel as shown in Figure 2. If N is 4, then we can create two threads to calculate the C matrix, where Thread 1 computes the elements of the first two rows and Thread 2 computes the elements of the next two rows in parallel. If we create four threads, then each thread computes elements of one row of C in parallel. The design idea is to distribute the work equally among the threads, when N is an exact multiple of the number of threads.

```
// parallel program to  multiply two nxn matrices- matrix_
mul_pthreads.c
#include <stdio.h>
#include <stdlib.h>
#define n 5120                    // global space
#define nthreads 2
int a[n][n],b[n][n],c[n][n];

void *threadfun(void *arg)        // Each Thread
Will do this.
{
    int *p=(int *)arg;
      int i,j,k;

         for(i=*p;i<(*p+(n/nthreads));i++)
          for(j=0;j<n;j++)
         for(k=0;k<n;k++)
           c[i][j]=c[i][j]+a[i][k]*b[k][j];

}

int main()
{

     int i,j,k,r,rownos[nthreads];

      pthread_t tid[nthreads];

        for(i=0;i<n;i++)     // Data
      Initialization.
    for(j=0;j<n;j++)
     {
         a[i][j]=1;
         b[i][j]=1;

    }

          // thread creations using pthreads API.
    for(i=0;i<nthreads;i++)
    {
        rownos[i]=i*(n/nthreads);
        pthread_create(&tid[i],NULL,threadfun,&rowno
s[i]);

    }

    // making main thread to wait until all other
threads complete using pthreads API.
       for(i=0;i<nthreads;i++)
            pthread_join(tid[i],NULL);

   /*printf("\n The result of multiplication is :\n");
     for(i=0;i<n;i++)
      {
        for(j=0;j<n;j++)
        printf(" %d",c[i][j]);

        printf("\n");
```

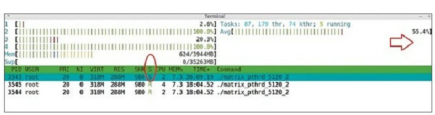

Figure 5: Multi threaded application with one boss and two worker threads

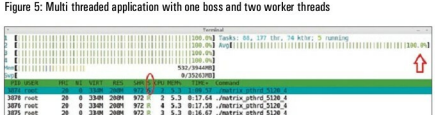

Figure 6: Multi threaded application with one boss and 4 worker threads